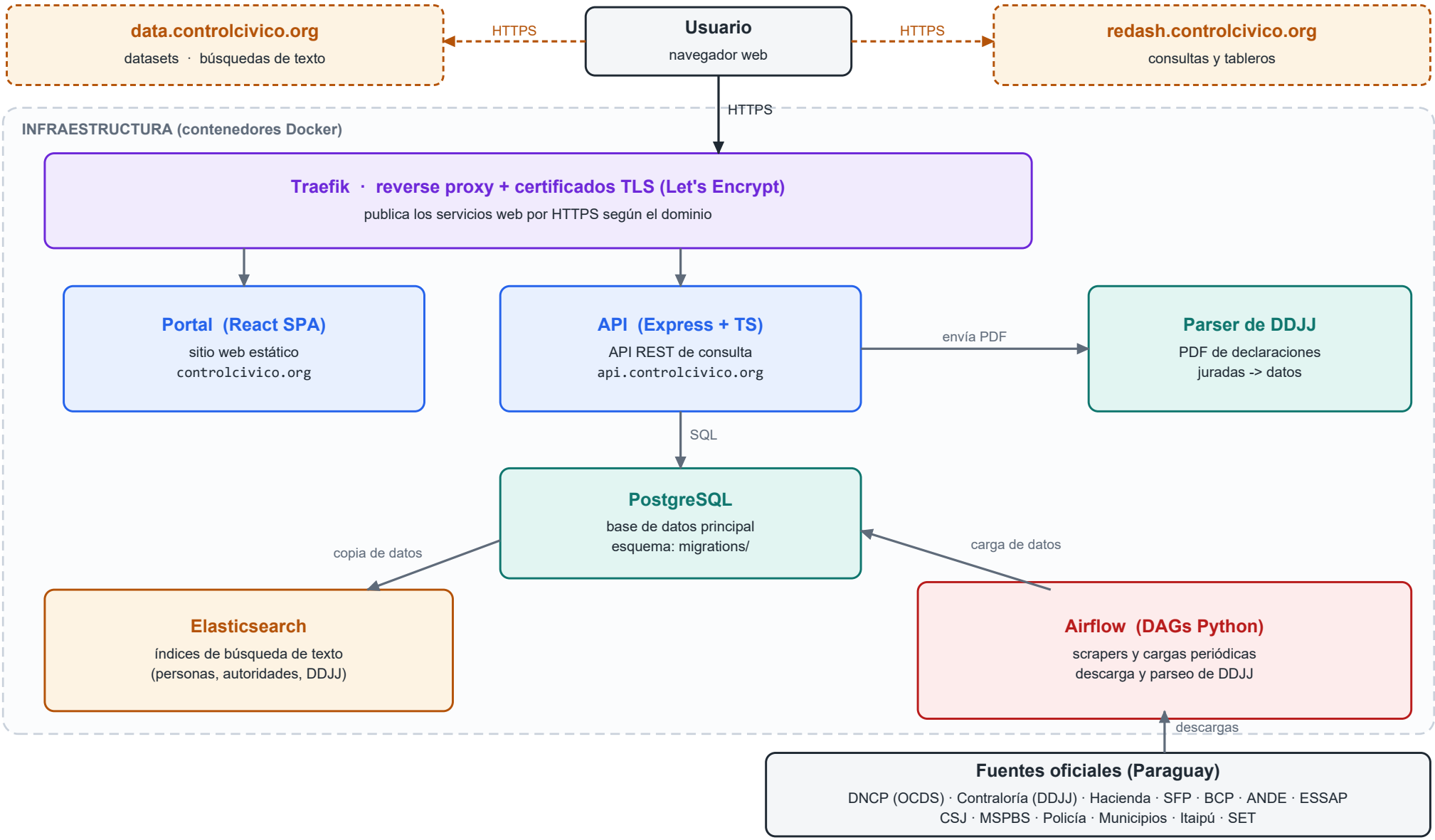
Control Cívico — Arquitectura

Portal de datos abiertos para el control de recursos públicos de Paraguay (InstIDEA). Vista general de los componentes y de cómo se relacionan.



Líneas punteadas: servicios a los que el navegador llama directamente.

1. Componentes del repositorio

Carpeta	Qué es	Tecnología	Cómo se ejecuta
portal/	Sitio web (SPA): compras COVID/OCDS, declaraciones juradas, autoridades, análisis de patrimonio	React, TypeScript, Ant Design, Nivo, ReactiveSearch	Imagen Docker con el build estático
api/	API REST: /api/ocds/*, /api/contralory/declarations, /api/people/:doc/declarations, /api/analysis/*	Node, Express, TypeScript, pg, Mocha	Contenedor detrás de Traefik
scripts/python/parser/	Microservicio que convierte PDF de declaraciones juradas en datos	FastAPI, pdftotext	Imagen microservice_parser
scripts/python/airflow/	DAGs: Contraloría (listado, descarga y parseo de DDJJ), CSJ, MSPBS, Policía, municipios, SFP, Itaipú…	Apache Airflow	Imagen dags
scripts/python/similarity/	Similitud entre proveedores (mismos datos de contacto)	Python	Manual
etl/	ETL legacy: ANDE, ESSAP, BCP, Hacienda, inflación, SET, Pytyvõ, Ñangareko	Pentaho Kettle (.ktr/.kjb)	Manual
migrations/	Esquema de la base (V1–V10 + scripts R_)	SQL	Manual
infra/	docker-compose de referencia: base, API, Traefik, parser, Elasticsearch, Airflow	Docker Compose	En el servidor

2. Recorrido de una visita

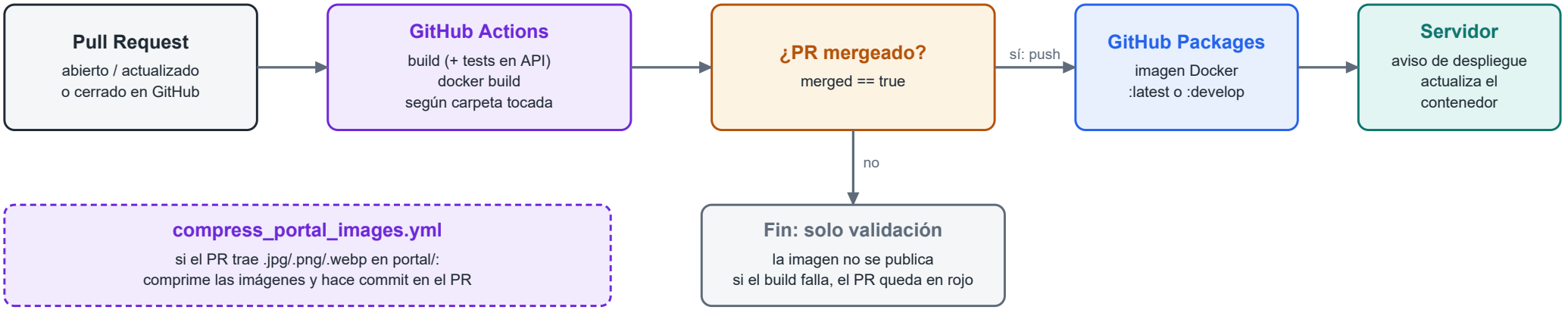
1. El navegador pide *controlcivico.org*. Traefik termina el HTTPS y entrega la SPA desde el contenedor del portal.
2. La SPA llama a *api.controlcivico.org*. La API Express consulta PostgreSQL y devuelve JSON.
3. Las búsquedas de texto (ReactiveSearch) van a Elasticsearch en *data.controlcivico.org*, que se alimenta con una copia de datos de PostgreSQL.
4. Las declaraciones juradas se procesan con el parser: convierte el PDF en activos, pasivos, patrimonio e ingresos, y el resultado se guarda en PostgreSQL.
5. Airflow corre periódicamente los scrapers de las fuentes oficiales y escribe en PostgreSQL.

3. Pipeline de declaraciones juradas (Contraloría)

Paso	DAG de Airflow	Qué hace	Dónde guarda
1. Listado	contralory_declaration_link_fetcher	Consulta el listado público de declaraciones de la Contraloría	staging.djbr_raw_data
2. Descarga	contralory_declaration_download_pdfs	Descarga el PDF de cada declaración y registra los errores	staging.djbr_downloaded_files · staging.djbr_download_files_errors
3. Parseo	contralory_declaration_data_extraction	Envía cada PDF al parser y guarda los totales; marca las inválidas	analysis.djbr_data · staging.djbr_invalid_files
4. Publicación	(manual)	Vista materializada que consume la API	analysis.djbr (REFRESH MATERIALIZED VIEW)

4. Qué pasa cuando se sube algo a GitHub (CI/CD)

Los workflows se disparan **solo con pull requests** que tocan su carpeta. Un push directo a master no despliega nada.



Workflow	Se dispara si el PR toca	En cada PR	Al mergear publica
portal.yml	portal/**	yarn, npm run build, docker build	imagen portal
api.yml	api/**	npm ci, tsc, npm test, docker build	imagen api
airflow.yml	scripts/python/airflow/**	docker build	imagen dags
microservice_parser.yml	scripts/python/parser/**	docker build	imagen microservice_parser
compress_portal_images.yml	imágenes .jpg/.png/.webp en portal/	comprime y hace commit en el PR	—
(ninguno)	etl/, migrations/, infra/, similarity/, dncp/	nada	Se aplica manualmente

Tags de imagen: master -> **latest**, develop -> **develop**. Después de publicar la imagen, el workflow avisa al servidor para que actualice el contenedor.

5. Comandos para desarrollo local

- Portal: `cd portal && yarn install && yarn start`
- Build y contenedor del portal: `yarn build && docker build -t portal .`
- API: `cd api && npm ci && npm start` · tests: `npm test`